

Complete DSA Learning Order

Topic Content Compilation

Section 11 — Dynamic Programming (11.1 – 11.7)

June 30, 2026

Section 11 — Dynamic Programming

11.1 — Identifying when a problem has the DP property

1. PLAIN-LANGUAGE GIST

Dynamic programming (DP) applies to problems that have two specific properties together: overlapping subproblems (the same smaller question gets asked multiple times during a naive recursive solution) and optimal substructure (the best answer to the big problem can be built directly from the best answers to its smaller subproblems, with no need to reconsider those subproblems differently). Recognizing both properties together is the entire skill this topic teaches — DP isn't a specific algorithm, it's a recognition pattern that unlocks a specific optimization.

2. REAL-WORLD ANALOGY

Picture calculating the total cost of building a house by first calculating the cost of building each room, where multiple rooms happen to need an identical sub-component (say, the same standard window installation) — if you calculate that window's cost once and reuse the answer for every room that needs it, instead of recalculating it fresh each time, you avoid massive redundant work. That reusability is overlapping subproblems. Separately, if the total house cost is genuinely just the sum of its rooms' costs (no hidden interactions between rooms changing the math), that's optimal substructure — the big answer builds cleanly from the smaller ones.

3. CONNECT TO PRIOR KNOWLEDGE

This topic formalizes something flagged twice already without a name: 0.3's naive Fibonacci recursion ($O(2^n)$ from repeated branching) and 5.1's explicit callout that problems needing two smaller subproblems combined (like the stairs-climbing example) are “an early signal to watch for overlapping/repeated subproblems” — this is exactly that signal, now given its formal definition and connected to the second required property (optimal substructure) that 5.1 didn't yet name.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Overlapping subproblems means that if you tried to solve the problem with plain recursion, you'd find yourself computing the exact same smaller input's answer multiple times across different branches of the recursion tree. Optimal substructure means the optimal solution to the full problem can be constructed directly and correctly from optimal solutions to its subproblems — there's no scenario where you need a suboptimal answer to a subproblem in order to build the truly optimal answer to the bigger problem.

Next layer — why both properties are needed, and how to test for them concretely:

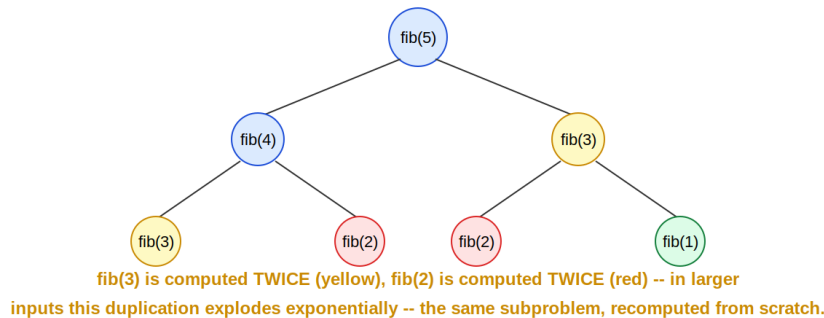
- **Testing for overlapping subproblems:** write out (or mentally trace, 0.8's skill) the brute-force recursive solution's call tree for a small input. If the same (input) combination appears as a call more than once across different branches, overlap exists — exactly as 0.8 and 0.3 demonstrated with `fib(2)` appearing twice in `fib(4)`'s trace.
- **Testing for optimal substructure:** ask “if I already knew the best answer for every smaller version of this problem, could I combine a fixed, small number of them (plus maybe one extra step) to get the best answer for the current size, with no need to re-decide anything about the smaller subproblems themselves?” If yes, optimal substructure holds.
- **Why overlap alone isn't enough:** a problem could have repeated subproblems that don't combine in a simple, reusable way (e.g., subproblems whose “optimal” answer depends on

some global context not captured by the subproblem's own state) — DP requires that each subproblem's optimal answer, once computed, is context-independent and reusable regardless of where it's needed.

• **Why optimal substructure alone isn't enough:** many recursive problems have clean optimal substructure (5.1's recursive design process applies broadly) but no actual overlap — in that case, plain recursion (or even a single pass) is already efficient, and the memoization/tabulation machinery (11.2/11.3) needed to exploit overlap would add complexity for zero benefit.

5. VISUAL REPRESENTATION

Overlapping subproblems: fib(5)'s call tree recomputes the same values repeatedly



Optimal substructure: fib(n)'s answer is built DIRECTLY from fib(n-1) and fib(n-2)'s answers -- no need to reconsider smaller subproblems differently.

BOTH properties together (overlap + optimal substructure) = DP applies.

Overlap alone without reusable substructure, or vice versa, isn't enough.

The diagram traces fib(5)'s call tree, highlighting that fib(3) and fib(2) are each computed twice from scratch — the overlapping subproblems property made visible. The accompanying note confirms the second property: fib(n)'s answer is built directly and cleanly from fib(n-1) and fib(n-2)'s answers, with no need to reconsider how those were computed — optimal substructure.

6. WHEN TO USE / WHEN NOT TO USE

Recognize the DP property when:

- A brute-force recursive solution's call tree visibly repeats identical subproblem calls (the concrete, checkable test from section 4) — this is the strongest, most reliable signal.
- The problem asks for an optimum (maximum, minimum, count of ways) over a sequence of choices, where each choice's "best so far" can be expressed purely in terms of a small, well-defined state (previewed across 11.4-11.6's specific patterns).

Wrong choice when:

- The recursive structure has no actual overlap (each subproblem is only ever asked once, as in most of 5.1's and 5.2's tree-shaped recursive/backtracking problems) — applying memoization here adds bookkeeping overhead without eliminating any redundant work, since there was none to eliminate.
- The problem lacks optimal substructure — e.g., certain problems where the best answer to a subproblem depends on additional context beyond that subproblem's own inputs (some graph problems with complex global constraints) — forcing a DP framing onto such a problem produces incorrect results, not just inefficient ones.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — verifying overlap by literally counting duplicate calls, the exact trace above:

```
Dictionary<int, int> callCounts = new();

int FibTraced(int n) {
    callCounts[n] = callCounts.GetValueOrDefault(n, 0) + 1; // 2.3
    if (n <= 1) return n;
    return FibTraced(n - 1) + FibTraced(n - 2);
}

// FibTraced(5); then inspect callCounts -- fib(3) called 2 times,
// fib(2) called 3 times, fib(1) called 5 times -- overlap CONFIRMED
```

This directly instruments the recursive function to count how many times each input value is actually requested — a concrete, runnable way to verify overlap exists rather than just reasoning about it abstractly.

INTERMEDIATE — a problem with optimal substructure but genuinely NO overlap, where DP machinery would be wasted effort:

```
// Sum of an array via recursion -- optimal substructure clearly holds
// (sum(arr[0..n]) = arr[n] + sum(arr[0..n-1])), but NO overlap --
// each sub-sum is asked for exactly once, ever.
int RecursiveSum(int[] arr, int index) {
    if (index < 0) return 0;
    return arr[index] + RecursiveSum(arr, index - 1);
}
// Adding a memo cache here would store n distinct results, each
// computed once, each looked up once -- zero redundant work avoided.
```

This is the explicit negative example: optimal substructure alone, without overlap, doesn't benefit from DP — recognizing this prevents over-engineering a perfectly efficient $O(n)$ recursive (or simple iterative) solution with unnecessary caching machinery.

ADVANCED — confirming both properties hold for a less obvious case (longest common subsequence, previewing 11.5), by reasoning about the recurrence directly:

```
// LCS(s1, s2) at positions (i, j): if s1[i] == s2[j], the answer
// extends LCS(i-1, j-1) by one character. Otherwise, it's the BETTER
// of LCS(i-1, j) or LCS(i, j-1).
//
// Optimal substructure: YES -- LCS(i,j)'s best answer is built purely
// from LCS(i-1,j-1), LCS(i-1,j), LCS(i,j-1)'s best answers.
//
// Overlapping subproblems: YES -- LCS(i-1,j) and LCS(i,j-1) themselves
// both eventually call LCS(i-1,j-1) again, deeper in their own
// recursion -- the same pair gets requested from different parents.
int LCSBrute(string s1, string s2, int i, int j) {
    if (i < 0 || j < 0) return 0;
    if (s1[i] == s2[j]) return 1 + LCSBrute(s1, s2, i - 1, j - 1);
    return Math.Max(LCSBrute(s1, s2, i - 1, j), LCSBrute(s1, s2, i, j - 1));
}
```

This shows the recognition process applied to a genuinely two-dimensional problem (state needs two indices, i and j , previewing 11.5) — both properties are verified by reasoning about the recurrence's structure directly, the same systematic check applied to fib, just with a richer state.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, formally proving optimal substructure for a new problem (rather than just informally checking it) typically uses a proof technique called an exchange argument or a cut-and-paste argument — showing that if a hypothetically “better” solution existed that didn't decompose into optimal subproblem solutions, you could always “cut out” the suboptimal

subproblem piece and “paste in” the truly optimal one, producing an equal-or-better overall solution, a contradiction that proves the property must hold. There's also a useful related concept worth knowing: memoization-friendly state design — sometimes a problem's naive recursive state (e.g., “the entire remaining string”) doesn't overlap because it's defined too broadly, but a cleverer, more compact state representation (e.g., “just the current index” rather than “the substring starting there”) reveals overlap that wasn't initially visible, turning an apparently non-DP problem into a genuine DP one through smarter state design.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming any recursive problem automatically qualifies for DP — many recursive problems (most of 5.1's and 5.2's examples) have zero actual overlap, and applying memoization to them is harmless but pointless overhead, not a meaningful optimization. Another is confusing “this problem can be solved recursively” with “this problem has optimal substructure” — recursion is a technique; optimal substructure is a property of the problem itself, and not every recursively-expressible problem has it. A third: stopping the analysis at “I found overlap” without separately verifying optimal substructure too — both properties are independently necessary, and a problem with overlap but no clean optimal substructure cannot be correctly solved via standard memoization/tabulation, no matter how tempting the apparent repeated-subproblem pattern looks.

10. SELF-CHECK

- What is the concrete, checkable test for whether a recursive solution has overlapping subproblems — describe it without saying “if it feels slow”?
- Why is optimal substructure alone (without overlapping subproblems) not a sufficient reason to apply memoization or tabulation?
- In the longest-common-subsequence brute-force example, walk through, out loud, exactly which specific subproblem call gets requested by two different parent calls, demonstrating the overlap concretely.

11.2 — Memoization (top-down DP)

1. PLAIN-LANGUAGE GIST

Memoization takes an existing brute-force recursive solution and adds a cache — a Dictionary mapping each unique input already computed to its result — checked at the very top of the function before any actual recursive work happens, so that any subproblem requested more than once (11.1's overlap condition) gets answered instantly from the cache instead of being recomputed from scratch.

2. REAL-WORLD ANALOGY

Picture a researcher answering the same frequently-asked question repeatedly for different visitors. Instead of redoing the full research from scratch every single time someone asks, they write the answer on a sticky note the first time, and for every subsequent visitor asking the same question, they just hand over the sticky note instead of researching again — the underlying “function” (answering the question) is unchanged, but a quick check against existing sticky notes short-circuits all the repeated work.

3. CONNECT TO PRIOR KNOWLEDGE

This is a direct, minimal-change application of 2.2's Dictionary<TKey,TValue> to the recursive functions you've already been writing throughout 5.1 and 11.1 — memoization doesn't require restructuring your recursive logic at all; it requires adding exactly one check at the top (cache lookup) and one write at the bottom (cache store) to an otherwise unchanged recursive function.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: At the very start of the recursive function, check whether the current input is already a key in a memo dictionary — if so, return the cached value immediately without doing any further work. Otherwise, proceed with the normal recursive logic as before, but just before returning the final computed result, store it in the memo dictionary keyed by the current input.

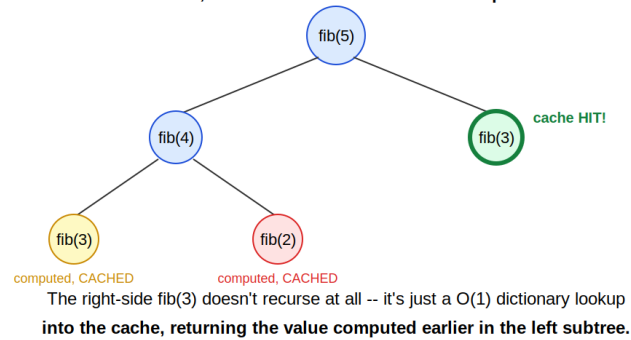
Next layer — why this transforms exponential blowup into linear-many unique calls, and the practical details:

- Each unique input value gets computed exactly once — the first time it's requested. Every subsequent request for that same input is an $O(1)$ (or $O(\log n)$ for some dictionary implementations, though C#'s Dictionary<TKey,TValue> is $O(1)$ average) lookup instead of a full recursive re-exploration.
- This turns naive Fibonacci's $O(2^n)$ exponential call count into $O(n)$ unique calls — exactly n distinct inputs (fib(0) through fib(n)) ever actually get computed, with every other call short-circuited by the cache, directly resolving the inefficiency 0.3 and 11.1 both flagged.
- **The cache key must capture every piece of information the function's behavior depends on** — if a recursive function takes multiple parameters (e.g., two indices, as in 11.1's LCS example), the memo key must encode both values together (a tuple, or a combined string/integer key), not just one of them; using an incomplete key causes the cache to incorrectly return a cached result for the wrong combination of inputs.
- **Memoization preserves the original recursive call structure** — unlike tabulation (11.3), which requires reorganizing the logic into an explicit loop with a determined fill order, memoization is a comparatively low-effort modification layered directly on top of existing brute-force recursive code, often making it the natural first step when converting a recognized DP problem (11.1) into an efficient solution.

5. VISUAL REPRESENTATION

Memoization: cache each subproblem's answer the FIRST time it's computed

fib(5) with a memo cache -- same call tree, but cache hits short-circuit recomputation



Top-down: still looks like plain recursion, but a Dictionary<input,result> check at the top of the function turns exponential blowup into linear-many UNIQUE calls.

The diagram traces the same fib(5) call tree from 11.1, now with a memo cache active: the right-side fib(3) subtree doesn't recurse at all — it's an instant cache hit, returning the value already computed and stored by the left-side fib(3) call moments earlier, visually showing exactly which branches get eliminated.

6. WHEN TO USE / WHEN NOT TO USE

Use memoization when:

- You've already confirmed both 11.1's DP properties hold, and you have (or are about to write) a brute-force recursive solution — memoization is the minimal-effort way to fix that solution's inefficiency without restructuring its logic.
- The natural recursive structure is easier to reason about than an equivalent iterative loop (some problems' recursive logic is significantly clearer than the corresponding bottom-up tabulation version) — memoization lets you keep that clarity while still achieving efficient performance.

Wrong choice when:

- The recursion depth could be large enough to risk a stack overflow (0.7's standing warning, directly applicable here) — tabulation (11.3) avoids this risk entirely by using an explicit loop with no recursive call stack at all.
- You need the absolute minimum possible memory usage, and the problem's structure allows a space-optimized tabulation approach (computing each state from only a small fixed window of prior states) — memoization's full dictionary of every unique input computed so far can use more memory than a carefully space-optimized iterative approach.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — memoized Fibonacci, the exact mechanism traced above:

```
Dictionary<int, long> memo = new();

long FibMemo(int n) {
    if (n <= 1) return n;
    if (memo.ContainsKey(n)) return memo[n];    // cache HIT

    long result = FibMemo(n - 1) + FibMemo(n - 2);
    memo[n] = result;                          // cache the result
    return result;
}
```

This is the exact same recursive logic as naive Fibonacci (0.3), with only two additions: the cache-check at the top, and the cache-store right before returning — no restructuring of the actual recursive relationship at all.

INTERMEDIATE — memoizing a function with a multi-part state (two indices), requiring a combined key, directly addressing 11.1's LCS preview:

```
Dictionary<(int, int), int> memo = new();

int LCSMemo(string s1, string s2, int i, int j) {
    if (i < 0 || j < 0) return 0;

    var key = (i, j); // combined key
    if (memo.ContainsKey(key)) return memo[key];

    int result;
    if (s1[i] == s2[j]) {
        result = 1 + LCSMemo(s1, s2, i - 1, j - 1);
    } else {
        result = Math.Max(LCSMemo(s1, s2, i - 1, j), LCSMemo(s1, s2, i, j - 1));
    }

    memo[key] = result;
    return result;
}
```

Using a C# tuple (i, j) as the dictionary key is a clean, idiomatic way to combine multiple pieces of state into one cache key — tuples have built-in equality and hashing support, making them a natural fit for Dictionary<TKey,TValue> (2.1's hashing mechanism) without needing a custom key type.

ADVANCED — memoization using an array instead of a Dictionary when the input range is small and known in advance:

```
long[] memoArray;

long FibMemoArray(int n) {
    if (n <= 1) return n;
    if (memoArray[n] != -1) return memoArray[n]; // -1 = "not computed"

    long result = FibMemoArray(n - 1) + FibMemoArray(n - 2);
    memoArray[n] = result;
    return result;
}

long FibSetup(int n) {
    memoArray = new long[n + 1];
    Array.Fill(memoArray, -1); // sentinel value meaning "uncached"
    return FibMemoArray(n);
}
```

When the possible inputs are bounded integers (like 0 to n), a plain array indexed directly by the input is faster than a Dictionary — avoiding hashing overhead entirely (2.1's deeper-knowledge point about fixed-range optimizations) — at the cost of needing to know the input range upfront and reserving that much array space regardless of how many values actually end up being computed.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, memoization generalizes directly into a broader programming concept called automatic memoization or function caching, available as a built-in language/library feature in

some environments (decorators in Python, certain caching attributes in other frameworks) — conceptually identical to what's shown here, just implemented as a reusable wrapper rather than hand-coded into each function. There's also a subtlety worth knowing: memoization assumes the function is pure (the same input always produces the same output, with no side effects or dependence on external mutable state) — applying memoization to a function that isn't pure produces incorrect cached results, since the cache has no way to know the “same” input might now warrant a different answer.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is using an incomplete or incorrect cache key for a multi-parameter recursive function — caching only by one of two relevant indices (as flagged in section 4) causes the function to incorrectly return a cached result computed for a different combination of inputs, a subtle bug that can silently produce wrong answers rather than crashing. Another is checking the cache after doing some of the recursive work rather than immediately at the top of the function — this partially defeats the purpose, since some redundant computation still happens before the cache check fires. A third: applying memoization to a recursive function with no actual overlapping subproblems (11.1's explicit negative example) — this adds dictionary overhead and complexity without eliminating any genuinely redundant work, since there was none to eliminate in the first place.

10. SELF-CHECK

- Why does memoization turn naive Fibonacci's exponential $O(2^n)$ call count into $O(n)$ unique computations — what specifically happens to the “duplicate” branches of the call tree?
- In the LCS example, why must the cache key combine both i and j together, rather than caching by just one of the two indices?
- Why does memoization assume the function being cached is “pure” (same input always produces the same output) — what would go wrong if this assumption were violated?

11.3 — Tabulation (bottom-up DP)

1. PLAIN-LANGUAGE GIST

Tabulation solves the same overlapping-subproblems-and-optimal-substructure problem as memoization, but works in the opposite direction: instead of starting from the original question and recursing down to base cases (top-down), tabulation starts from the base cases and builds up toward the final answer using an explicit loop and a table (typically an array), filling in each cell using only previously-filled cells, with no recursion at all.

2. REAL-WORLD ANALOGY

Picture filling in a row of dominoes' values where each domino's value is the sum of the two before it (a Fibonacci-style sequence) — instead of starting from the last domino and asking “what do I need to know to compute you?” (which is what recursion does, working backward), you simply start writing values into the first two dominoes directly, then move left to right, computing each subsequent domino's value the moment its two predecessors are already filled in.

3. CONNECT TO PRIOR KNOWLEDGE

This directly parallels 11.2's memoization in purpose (both exploit 11.1's DP properties to avoid redundant computation) while taking the opposite direction — and the table itself is just an application of 1.1's array indexing, with the “recurrence relation” (how each value relates to prior ones) playing the same role 0.3's loop-reading skill already trained you to recognize, just now used to build a solution rather than just analyze one.

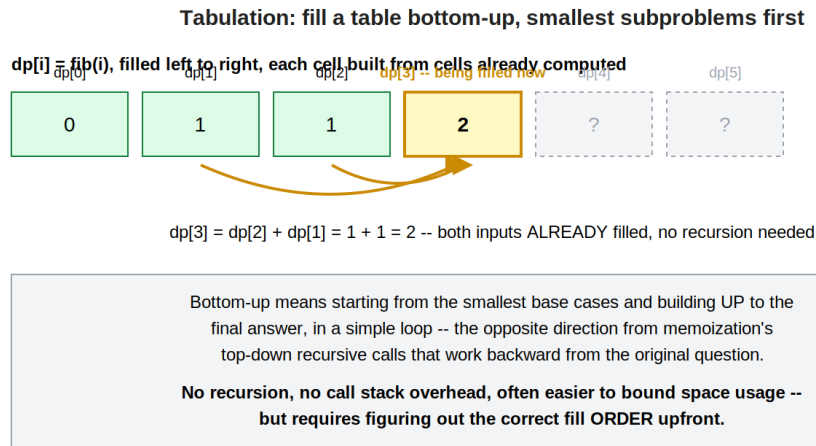
4. CORE MECHANISM (Simple → Intermediate)

Simple model: Create a table (usually an array `dp[]`) sized to hold every subproblem's answer. Fill in the base case(s) directly (no computation needed — they're given). Then loop through the remaining table positions in an order that guarantees every value a cell depends on is already filled by the time you reach it, computing each cell using the recurrence relation (the same relationship that defined the recursive case in a memoized or brute-force version).

Next layer — why determining fill order matters, and the relationship to memoization's recursive structure:

- **The fill order must respect dependencies:** for Fibonacci, `dp[i]` depends on `dp[i-1]` and `dp[i-2]`, both smaller indices — so filling the table left-to-right (increasing index) guarantees both dependencies are always already computed. For 11.5's 2D grid problems, this might mean filling row by row, or column by column, or diagonally, depending on which directions the recurrence actually depends on.
- **Tabulation's recurrence relation is identical to memoization's recursive case** — converting a memoized solution into a tabulated one (formalized in 11.7) means taking the same “how does this state relate to smaller states” logic and re-expressing it as a loop's update rule instead of a recursive call, with the base cases becoming the table's initial values instead of the recursion's stopping condition.
- **No call stack is used at all** — tabulation is purely iterative, completely eliminating the stack-overflow risk that recursion-based memoization carries on very large inputs (0.7's recurring warning), which is tabulation's most concrete practical advantage.
- **Tabulation often makes space optimization easier to spot** — once you can see the entire table being filled explicitly, it's often visually obvious that only a small number of previous rows/cells are ever actually needed at once, directly motivating 11.7's final “space optimization” conversion step.

5. VISUAL REPRESENTATION



The diagram shows a Fibonacci table being filled left to right: $dp[3]$ is computed directly from $dp[2]$ and $dp[1]$, both already filled in, using a simple addition — no recursive calls, no call stack, just a loop reading from positions already computed earlier in the same pass.

6. WHEN TO USE / WHEN NOT TO USE

Use tabulation when:

- You need to avoid recursion's stack-overflow risk on a large input, or specifically want to demonstrate the iterative mechanism explicitly (an interviewer's request, similar to 4.1/10.4's iterative-vs-recursive distinction for stacks and DFS).
- You anticipate wanting to space-optimize the solution afterward (11.7) — having the explicit table makes it easier to see which prior values are genuinely still needed, simplifying that next optimization step.

Wrong choice when:

- The recursive structure is significantly clearer to reason about and write correctly than the equivalent iterative loop, and recursion depth isn't a practical concern for the given input size — in that case, memoization (11.2) may be the more naturally expressed, equally efficient choice.
- The problem's dependency structure is unusual or hard to determine a valid fill order for — some DP problems have recurrences that don't map cleanly onto a simple “smaller index first” loop, making memoization's automatic dependency resolution more straightforward.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — tabulated Fibonacci, the exact mechanism traced above:

```
long FibTabulation(int n) {
    if (n <= 1) return n;

    long[] dp = new long[n + 1];
    dp[0] = 0;           // base case
    dp[1] = 1;           // base case

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2]; // recurrence, as a loop
    }
    return dp[n];
}
```

INTERMEDIATE — tabulated version of 11.2's two-index LCS problem, requiring a 2D table and a carefully chosen fill order:

```
int LCSTabulation(string s1, string s2) {
    int m = s1.Length, n = s2.Length;
    int[,] dp = new int[m + 1, n + 1];    // shifted indices, 0.4-style

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i, j] = 1 + dp[i - 1, j - 1];
            } else {
                dp[i, j] = Math.Max(dp[i - 1, j], dp[i, j - 1]);
            }
        }
    }
    return dp[m, n];
}
```

Filling row by row, left to right, guarantees $dp[i-1, j-1]$, $dp[i-1, j]$, and $dp[i, j-1]$ are all already computed by the time $dp[i, j]$ is reached — directly mirroring the dependency structure from 11.2's memoized recursive version, just expressed as nested loops instead of recursive calls.

ADVANCED — tabulation with explicit reconstruction of the actual chosen path/sequence, a common, practically useful extension:

```
(int length, string sequence) LCSWithReconstruction(string s1, string s2) {
    int m = s1.Length, n = s2.Length;
    int[,] dp = new int[m + 1, n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            dp[i, j] = (s1[i - 1] == s2[j - 1])
                ? 1 + dp[i - 1, j - 1]
                : Math.Max(dp[i - 1, j], dp[i, j - 1]);
        }
    }

    // walk BACKWARD through the filled table to reconstruct the LCS
    var sb = new StringBuilder();
    int x = m, y = n;
    while (x > 0 && y > 0) {
        if (s1[x - 1] == s2[y - 1]) {
            sb.Insert(0, s1[x - 1]);    // part of LCS -- prepend it
            x--; y--;
        } else if (dp[x - 1, y] >= dp[x, y - 1]) {
            x--;
        } else {
            y--;
        }
    }
    return (dp[m, n], sb.ToString());
}
```

This shows a genuinely useful pattern beyond just computing the optimal value: once the full table exists, walking backward through it (using the same comparisons that built it forward) reconstructs the actual optimal sequence of choices — a common follow-up requirement that tabulation's fully-materialized table supports naturally, since every intermediate decision is still available to inspect after the fact.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, some DP problems have dependency structures complex enough that a simple nested-loop fill order isn't obvious — in those cases, a technique called DP over a DAG treats each subproblem as a node and each dependency as a directed edge, then uses 10.7's topological sort to determine a valid fill order automatically, generalizing the “smaller index first” intuition into a formal, provably correct ordering procedure for arbitrarily complex dependency structures. There's also a connection worth flagging to 11.2: a hybrid approach sometimes called “memoization with iterative driving” explicitly fills the table using a loop but still uses the recursive recurrence relation's logic directly inside that loop, blurring the line between top-down and bottom-up styles.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is choosing an incorrect fill order that processes a cell before one of its dependencies has been computed — this produces silently wrong results (reading a default/zero value instead of the actual needed prior computation) rather than an obvious crash, making it a particularly dangerous class of bug specific to tabulation. Another is forgetting to correctly initialize the base cases before the main loop begins — if $dp[0]$ and $dp[1]$ aren't set correctly before the loop starts reading from them, every subsequent cell inherits that initial error. A third: assuming tabulation is always strictly “better” than memoization because it avoids recursion — for problems where determining the correct fill order is itself difficult or error-prone, memoization's automatic dependency resolution can be the more reliable, easier-to-get-correct choice, even at the cost of recursion's stack-overflow risk on very large inputs.

10. SELF-CHECK

- Why must a tabulated solution's fill order guarantee that every cell's dependencies are already computed before that cell is reached — what specifically goes wrong if this isn't respected?
- Walk through, out loud, why filling the LCS table row by row, left to right, correctly guarantees all three of $dp[i-1, j-1]$, $dp[i-1, j]$, and $dp[i, j-1]$ are available when computing $dp[i, j]$.
- Why does tabulation make it easier to reconstruct the actual optimal sequence of choices (not just the optimal value), compared to a memoized recursive approach?

11.4 — 1D DP patterns (e.g. climbing stairs, house robber)

1. PLAIN-LANGUAGE GIST

A 1D DP problem is one whose state can be fully described by a single index or position — $dp[i]$ represents “the best answer considering only the first i elements” — and each cell depends on only a small, fixed number of previous cells along that same single dimension, making this the simplest, most foundational shape of dynamic programming problem to recognize and implement.

2. REAL-WORLD ANALOGY

Picture walking up a staircase where, at each step, your “best strategy so far” can be fully summarized by just two numbers: how many ways you could have reached the step just below you, and how many ways you could have reached the step two below that. You never need to remember anything about steps further back than that — the entire relevant history collapses into a tiny, fixed-size summary at each position.

3. CONNECT TO PRIOR KNOWLEDGE

This is the direct generalization of 5.1's stairs-climbing example (which explicitly previewed needing two combined recursive subproblems) and 0.3's naive Fibonacci, now formalized as a pattern category — 1D DP problems are precisely the ones where 11.1's “fixed, small number of subproblems” optimal-substructure test resolves to a single index, rather than the two-index state that 11.5's 2D patterns will require.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Define $dp[i]$ as the answer to the problem considering only the first i elements (or up to position i). Identify the recurrence: how does $dp[i]$ relate to $dp[i-1]$, $dp[i-2]$, or some other small, fixed set of earlier positions? Fill the table (or compute recursively with memoization) using that recurrence, starting from clearly-defined base cases.

Next layer — the recognizable shapes within 1D DP, and how to distinguish them:

- **“Counting ways” problems** (climbing stairs: how many distinct ways to reach step n , taking 1 or 2 steps at a time): the recurrence is typically a sum of contributions from each smaller subproblem — $dp[i] = dp[i-1] + dp[i-2]$, since every way to reach step $i-1$ (then take one more step) and every way to reach step $i-2$ (then take a two-step) are both valid, distinct paths to step i .
- **“Optimal value” problems** (house robber: maximum money robbable without robbing two adjacent houses): the recurrence is typically a max (or min) over a small set of choices — $dp[i] = \max(dp[i-1], dp[i-2] + \text{value}[i])$, representing the binary choice “skip this position” versus “take this position, which then forces skipping the previous one.”
- **Recognizing which shape applies:** “how many ways” language signals summing contributions; “maximum/minimum value” language signals taking a max/min over a small set of choice-driven recurrences — this distinction in the problem's own wording is a strong, practical signal for which recurrence shape to reach for first.
- The state being just one index is what makes these “1D” — contrast directly with 11.5's 2D patterns, where the state needs two coordinates to be fully described.

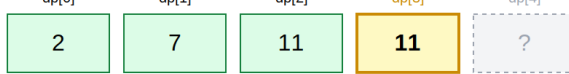
5. VISUAL REPRESENTATION

1D DP: House Robber -- each $dp[i]$ depends on just a couple PRIOR cells

Houses (money): [2, 7, 9, 3, 1] -- can't rob two ADJACENT houses



$dp[i]$ = max money robbable using houses 0..i



$$\begin{aligned} dp[3] &= \max(dp[2] \text{ (skip house 3)}, dp[1] + \text{house}[3] \text{ (rob house 3)}) \\ &= \max(11, 7 + 3) = \max(11, 10) = 11 \end{aligned}$$

Only $dp[i-1]$ and $dp[i-2]$ are ever needed -- the "1D" in 1D DP means each cell depends on a SMALL FIXED WINDOW of prior cells, not the entire table so far.

The diagram traces House Robber's dp table being filled: $dp[3]$ is computed as $\max(dp[2], dp[1] + \text{house}[3])$ — explicitly showing the binary "skip or take" choice recurrence, and that only $dp[i-1]$ and $dp[i-2]$ are ever consulted, never the full table, which is precisely the "1D" signature.

6. WHEN TO USE / WHEN NOT TO USE

Recognize a 1D DP pattern when:

- The problem involves a single sequence (an array, a string, a number line of positions), and the answer at each position can be expressed purely in terms of a small, fixed number of earlier positions in that same sequence.
- The problem explicitly involves counting distinct ways to reach a state, or optimizing (max/min) a value accumulated while making sequential choices along a single sequence.

Wrong choice when:

- The problem genuinely requires tracking position within two independent sequences or dimensions simultaneously (comparing two strings, navigating a 2D grid) — that's 2D pattern instead, and forcing a 1D state onto such a problem loses information the recurrence actually needs.
- The "best so far" at each position depends on more than just a small fixed window of prior positions — if the recurrence genuinely needs to look arbitrarily far back, a plain 1D array-indexed DP may need restructuring, or the problem may not have clean 1D optimal substructure at all.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — climbing stairs, the "counting ways" recurrence shape:

```
int ClimbStairs(int n) {
    if (n <= 2) return n;    // base cases

    int[] dp = new int[n + 1];
    dp[1] = 1;
    dp[2] = 2;
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];    // SUM -- both ways count
    }
    return dp[n];
}
```

INTERMEDIATE — house robber, the "optimal value" max-recurrence shape, the exact diagram above:

```

int HouseRobber(int[] houses) {
    if (houses.Length == 0) return 0;
    if (houses.Length == 1) return houses[0];

    int[] dp = new int[houses.Length];
    dp[0] = houses[0];
    dp[1] = Math.Max(houses[0], houses[1]);

    for (int i = 2; i < houses.Length; i++) {
        dp[i] = Math.Max(dp[i - 1], dp[i - 2] + houses[i]); // skip/take
    }
    return dp[houses.Length - 1];
}

```

ADVANCED — a variant combining both recurrence shapes: counting the number of ways to achieve the maximum value, requiring tracking two parallel 1D tables simultaneously:

```

(int maxValue, int wayCount) HouseRobberCountWays(int[] houses) {
    int n = houses.Length;
    if (n == 0) return (0, 1);

    int[] dpValue = new int[n];
    int[] dpCount = new int[n];

    dpValue[0] = houses[0];
    dpCount[0] = 1;

    if (n > 1) {
        if (houses[1] > houses[0]) {
            dpValue[1] = houses[1]; dpCount[1] = 1;
        } else if (houses[1] < houses[0]) {
            dpValue[1] = houses[0]; dpCount[1] = 1;
        } else {
            dpValue[1] = houses[0]; dpCount[1] = 2; // TIE
        }
    }

    for (int i = 2; i < n; i++) {
        int takeOption = dpValue[i - 2] + houses[i];
        int skipOption = dpValue[i - 1];

        if (takeOption > skipOption) {
            dpValue[i] = takeOption; dpCount[i] = dpCount[i - 2];
        } else if (skipOption > takeOption) {
            dpValue[i] = skipOption; dpCount[i] = dpCount[i - 1];
        } else {
            dpValue[i] = takeOption; // TIE
            dpCount[i] = dpCount[i - 2] + dpCount[i - 1]; // both count
        }
    }
    return (dpValue[n - 1], dpCount[n - 1]);
}

```

This combines the “max value” recurrence with a parallel “counting ways” recurrence tracked alongside it — when the take/skip options tie exactly, both paths contribute to the way-count, a genuinely subtle extension showing that 1D DP patterns can be layered rather than always being a single isolated value per cell.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, many 1D DP problems can be reformulated using matrix exponentiation to compute $dp[n]$ in $O(\log n)$ time instead of $O(n)$ — representing the recurrence relation as a matrix multiplication, then using fast exponentiation (conceptually related to 0.3’s “doubling” intuition) to

compute the n -th term without iterating through every intermediate value. There's also a useful generalization: Kadane's algorithm (1.7's maximum-subarray-sum technique) is itself a 1D DP pattern in disguise — $dp[i]$ representing “best subarray sum ending exactly at position i ” — recognizing this connection retroactively reframes an earlier-learned greedy-feeling trick as a specific instance of this topic's broader pattern category.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is confusing the “counting ways” (sum-based) and “optimal value” (max/min-based) recurrence shapes — applying a sum where a max was needed (or vice versa) produces a structurally wrong answer, not just an inefficient one. Another is forgetting to handle small base cases explicitly and correctly (arrays of length 0 or 1) before the main loop begins — a loop starting at index 2 silently produces wrong or crashing behavior on inputs too short to ever reach that starting index. A third: not recognizing when a problem actually has TWO interacting quantities per position and trying to force a single 1D array to track both simultaneously, rather than maintaining two parallel tables that reference each other correctly.

10. SELF-CHECK

- What's the practical difference in problem wording that signals “use a sum-based recurrence” versus “use a max/min-based recurrence” in a 1D DP problem?
- Why does House Robber's recurrence need to look back at both $dp[i-1]$ AND $dp[i-2]$, rather than just $dp[i-1]$ alone?
- In the “count ways to achieve max value” advanced example, why does a tie between the take and skip options require adding the two options' way-counts together, rather than just picking one arbitrarily?

11.5 — 2D DP patterns (e.g. grid paths, longest common subsequence)

1. PLAIN-LANGUAGE GIST

A 2D DP problem is one whose state needs two independent coordinates to fully describe — $dp[i][j]$ typically represents “the best answer considering the first i elements of one thing and the first j elements of another” (or a position (row, col) in an actual 2D grid) — generalizing 11.4's single-index 1D pattern into a full grid of interdependent subproblems.

2. REAL-WORLD ANALOGY

Picture comparing two family trees to find the longest shared lineage — at any point in the comparison, “how far have I matched so far” genuinely needs two separate position trackers (where you are in tree A, and where you are in tree B), since progress in one tree doesn't automatically tell you anything about progress in the other. That two-coordinate tracking requirement is exactly what makes a problem 2D rather than 1D.

3. CONNECT TO PRIOR KNOWLEDGE

This directly extends 11.1's LCS preview and 11.4's 1D foundation — the core DP recognition skill and the memoization/tabulation mechanisms (11.2/11.3) are unchanged; only the shape of the state changes, from a single index to a pair, mirroring how 1.8's matrix traversal generalized 1.1's single-dimension array indexing into two dimensions.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Define $dp[i][j]$ as the answer considering the first i elements of one sequence/dimension and the first j elements of the other. Identify how $dp[i][j]$ relates to nearby cells — commonly $dp[i-1][j]$, $dp[i][j-1]$, and/or $dp[i-1][j-1]$ — and fill the table in an order (row by row, or column by column) guaranteeing those dependencies are always ready.

Next layer — the two main flavors of 2D DP, and the dependency directions each one needs:

- **Two-sequence comparison problems** (longest common subsequence, edit distance): $dp[i][j]$ compares the first i characters of string A against the first j characters of string B. The recurrence typically checks “do the current characters match?” — if so, extend $dp[i-1][j-1]$; if not, take the best of $dp[i-1][j]$ or $dp[i][j-1]$, exactly 11.1's LCS recurrence formalized as a named pattern category.
- **Grid traversal problems** (unique paths, minimum path sum): $dp[r][c]$ represents the answer for reaching cell (r, c) in an actual 2D grid (1.8's structure). The recurrence typically combines $dp[r-1][c]$ (arrived from above) and $dp[r][c-1]$ (arrived from the left), since those are the only two cells that could lead into (r, c) under a “right or down only” movement constraint.
- **The fill order must respect whichever specific dependencies the recurrence has** — for both flavors above, filling row by row, left to right, naturally ensures $dp[i-1][*]$ and $dp[i][j-1]$ are already computed, but a different recurrence shape would need filling in the opposite direction instead — always trace the specific dependency arrows before committing to a fill order.
- 2D DP's space cost is naturally $O(\text{rows} \times \text{cols})$, a direct consequence of needing a full grid rather than a single array — this is precisely why 11.7's space-optimization step is a particularly common and valuable optimization specifically for 2D DP problems.

5. VISUAL REPRESENTATION

2D DP: unique paths through a grid, each cell depends on TWO directions

$dp[r][c]$ = number of ways to reach cell (r,c) moving only right or down

1	1	1	1
1	2	3	4
1	3	6	?

from LEFT ($dp[1][2]=3$)

$$dp[1][3] = dp[0][3] \text{ (from above)} + dp[1][2] \text{ (from left)} = 1 + 3 = 4$$

The "2D" in 2D DP means the state needs TWO indices to describe (row AND column) -- each cell combines results from two specific other cells (above, left), generalizing 1D DP's single-direction dependency into a 2D grid of states.

The diagram traces the "unique paths" grid problem: $dp[1][3]$ is computed by summing the cell above ($dp[0][3]$) and the cell to the left ($dp[1][2]$) — the two-direction dependency that defines this entire pattern category, generalizing 11.4's single-direction $dp[i-1]$ lookup into two simultaneous directions.

6. WHEN TO USE / WHEN NOT TO USE

Recognize a 2D DP pattern when:

- The problem involves comparing or combining two sequences simultaneously (two strings, two arrays) where progress through each needs independent tracking.
- The problem is literally framed on a 2D grid (1.8's structure) where movement or accumulation happens across two spatial dimensions (rows and columns).

Wrong choice when:

- The problem's state genuinely collapses to a single index once you look closely — always check whether a full 2D state is actually necessary before defaulting to it.
- The relationship between the two dimensions isn't actually about combining/comparing them in the ways described above — forcing a $dp[i][j]$ framing onto a problem whose real structure doesn't match either flavor risks an incorrect recurrence relation, not just unnecessary complexity.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — unique paths in a grid, the "grid traversal" flavor, the exact diagram above:

```
int UniquePaths(int rows, int cols) {
    int[,] dp = new int[rows, cols];

    for (int r = 0; r < rows; r++) dp[r, 0] = 1; // base: first column
    for (int c = 0; c < cols; c++) dp[0, c] = 1; // base: first row

    for (int r = 1; r < rows; r++) {
        for (int c = 1; c < cols; c++) {
            dp[r, c] = dp[r - 1, c] + dp[r, c - 1]; // above + left
        }
    }
    return dp[rows - 1, cols - 1];
}
```

INTERMEDIATE — longest common subsequence, the "two-sequence comparison" flavor, directly tabulating 11.1/11.2's recurrence:

```

int LCS(string s1, string s2) {
    int m = s1.Length, n = s2.Length;
    int[,] dp = new int[m + 1, n + 1];    // shifted indices, 0.4-style

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            dp[i, j] = (s1[i - 1] == s2[j - 1])
                ? 1 + dp[i - 1, j - 1]
                : Math.Max(dp[i - 1, j], dp[i, j - 1]);
        }
    }
    return dp[m, n];
}

```

ADVANCED — edit distance, a more elaborate two-sequence comparison requiring THREE possible transition directions:

```

int EditDistance(string s1, string s2) {
    int m = s1.Length, n = s2.Length;
    int[,] dp = new int[m + 1, n + 1];

    for (int i = 0; i <= m; i++) dp[i, 0] = i;    // delete all of s1[0..i]
    for (int j = 0; j <= n; j++) dp[0, j] = j;    // insert all of s2[0..j]

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1[i - 1] == s2[j - 1]) {
                dp[i, j] = dp[i - 1, j - 1];    // match, no cost
            } else {
                dp[i, j] = 1 + Math.Min(
                    dp[i - 1, j],    // delete from s1
                    Math.Min(
                        dp[i, j - 1],    // insert into s1
                        dp[i - 1, j - 1] // substitute
                    )
                );
            }
        }
    }
    return dp[m, n];
}

```

This is structurally the same 2D “two-sequence comparison” template as LCS, but the recurrence considers three possible prior cells (not two) when characters don’t match, each representing a different edit operation (delete, insert, substitute) — showing that the 2D pattern category accommodates richer recurrences than the simplest LCS case, while keeping the same fundamental “two indices, dependency on nearby cells” shape.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, some 2D DP problems generalize further into 3D (or higher-dimensional) DP, where the state needs three or more independent coordinates (e.g., comparing two strings while also tracking a count of some additional constraint, like “at most k modifications allowed”) — the same core recognition and tabulation principles apply, just with correspondingly more nested loops and more carefully traced dependency directions. There’s also a valuable connection worth flagging: many 2D grid DP problems are secretly shortest-path problems in disguise (minimum path sum is literally a shortest-path computation on a DAG-shaped grid), connecting this topic back to 10.3’s BFS and previewing why Tier 2’s Dijkstra’s algorithm and DP sometimes solve overlapping classes of problems through genuinely different mechanisms.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is assuming a 2D-looking problem genuinely needs a full 2D table when a more careful look reveals the state can collapse into a 1D pattern (11.4) with less space overhead — not every problem involving two sequences or a grid actually requires tracking both dimensions independently in the final state. Another is getting the fill order wrong relative to the actual recurrence's dependencies — defaulting to “always fill row by row, left to right” without checking whether the specific recurrence relation actually depends on cells in that direction. A third: forgetting the base case initialization for an entire row or column (not just a single cell) — 2D DP base cases are frequently a whole edge of the table, and missing this full-edge initialization leaves part of the table reading uninitialized (default zero) values that propagate incorrect results throughout the rest of the table.

10. SELF-CHECK

- Why does the “unique paths” grid problem's recurrence depend on exactly two prior cells (above and left), while edit distance's recurrence depends on three?
- Walk through, out loud, why filling a 2D DP table row by row, left to right, correctly satisfies the dependency requirements for both the LCS and unique-paths recurrences specifically.
- Why might a problem that initially looks like it needs a 2D state actually be solvable with a simpler 1D approach — what would you check to determine this before committing to the more complex 2D table?

11.6 — Knapsack pattern (0/1 knapsack and variations)

1. PLAIN-LANGUAGE GIST

The 0/1 knapsack pattern solves “given a set of items, each with a weight and a value, and a maximum capacity, choose a subset of items maximizing total value without exceeding capacity” — and the “0/1” specifically means each item can be used at most once (included or excluded, a binary choice), distinguishing it from variants that allow reusing items.

2. REAL-WORLD ANALOGY

Picture packing a suitcase with a strict weight limit before a flight, where you have a pile of items each with a weight and a sentimental/practical value, and you can only pack a single copy of each item (no duplicates available) — you need to choose which subset maximizes total value while staying under the weight limit, the exact 0/1 knapsack scenario.

3. CONNECT TO PRIOR KNOWLEDGE

This is a specific, named, extremely common instance of 11.5's 2D DP pattern — the state needs two coordinates (which items have been considered so far, and how much capacity remains), and the recurrence is precisely 5.4's “include or exclude” binary choice (the same structural decision subset generation made at every element), now combined with a capacity constraint and an optimization objective rather than just enumeration.

4. CORE MECHANISM (Simple → Intermediate)

Simple model: Define $dp[i][w]$ as the best total value achievable using only the first i items, with a knapsack capacity of w . For each item, at each capacity, decide: exclude this item ($dp[i][w] = dp[i-1][w]$, value unchanged, capacity unchanged) or include this item, if it fits ($dp[i][w] = value[i] + dp[i-1][w - weight[i]]$, gaining this item's value but consuming its weight from the remaining capacity, looked up using only the previous items). Take whichever option gives the higher value.

Next layer — why “0/1” matters, and the common variations that change this one detail:

- The “0/1” constraint is enforced specifically by referencing $dp[i-1]$ (the previous item row) when including an item, rather than $dp[i]$ (the current item row) — this guarantees the same item can't be “included” multiple times within the same capacity calculation, since including it moves you to a different item's row, never back to itself.
- **Unbounded knapsack** (items can be reused infinitely) changes this single detail: the include-branch references $dp[i][w - weight[i]]$ (the same item row, not $i-1$), allowing the same item to be included again and again as long as capacity permits — a small but structurally meaningful change to which row the recurrence looks back into.
- **Bounded knapsack** (each item has a limited quantity, more than 1 but not unlimited) sits between these two extremes, typically handled by either explicitly looping over “how many of this item to include” at each step, or by a more advanced binary-decomposition trick.
- The “knapsack” framing generalizes far beyond literal weight/value problems — any problem with “select a subset of items, each used at most once, to optimize a value subject to a capacity-style constraint” fits this exact pattern, even when the surface story has nothing to do with physical packing (e.g., “partition a set into two subsets with minimum difference” is a disguised knapsack variant).

5. VISUAL REPRESENTATION

0/1 Knapsack: each item, each capacity -- include or exclude?

Items: (weight=2,val=3), (weight=3,val=4), (weight=4,val=5). Capacity = 5

$dp[i][w]$ = best value using first i items, with capacity w

	w=0	w=1	w=2	w=3	w=4	w=5
item 1 (w=2)	0	0	3	3	3	3
item 2 (w=3)	0	0	3	4	4	7

At $dp[2][5]$: EXCLUDE item2 -> $dp[1][5]=3$. INCLUDE item2 -> $v(4) + dp[1][5-3]=dp[1][2]=3$ -> 7
 Take the MAX: $\max(3, 7) = 7$ -- including item 2 wins here

Every cell asks the SAME binary question: include this item (add its value, subtract its weight from capacity, look up the remaining capacity's best value using items BEFORE this one) or exclude it (just copy the row above)?

"0/1" means each item is used at most ONCE -- this binary include/exclude choice, repeated across all items and capacities, IS the 2D DP table.

(Unbounded knapsack variants allow reusing items -- a small but important twist.)

The diagram traces filling $dp[2][5]$ (considering the first 2 items, capacity 5): excluding item 2 gives $dp[1][5] = 3$; including item 2 gives its value (4) plus $dp[1][2]$ (the best value using only item 1, with the remaining capacity after subtracting item 2's weight) — taking the better of the two options, 7, directly visualizing the binary include/exclude decision at the heart of this pattern.

6. WHEN TO USE / WHEN NOT TO USE

Use the 0/1 knapsack pattern when:

- The problem describes selecting a subset of distinct items (each usable at most once) to optimize some value, subject to a capacity-style constraint (weight, budget, count) — the defining shape of this entire pattern family.
- A problem's surface description doesn't mention “knapsack” at all but matches the same underlying shape — recognizing the disguised form is often the harder, more valuable skill than implementing the well-known literal version.

Wrong choice when:

- Items can genuinely be reused multiple times without limit — that's unbounded knapsack, requiring the single-detail recurrence change described in section 4, not the standard 0/1 version.
- The problem doesn't actually have a capacity-style constraint at all — that's typically a much simpler optimization, and forcing a full knapsack DP table onto it is unnecessary overhead.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — classic 0/1 knapsack, the exact diagram and recurrence above:

```

int Knapsack01(int[] weights, int[] values, int capacity) {
    int n = weights.Length;
    int[, ] dp = new int[n + 1, capacity + 1];

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i, w] = dp[i - 1, w];           // EXCLUDE item i
            if (weights[i - 1] <= w) {
                dp[i, w] = Math.Max(dp[i, w],
                    values[i - 1] + dp[i - 1, w - weights[i - 1]]); // INCLUDE
            }
        }
    }
    return dp[n, capacity];
}

```

INTERMEDIATE — unbounded knapsack, the single-detail change (referencing $dp[i]$, not $dp[i-1]$, when including):

```

int KnapsackUnbounded(int[] weights, int[] values, int capacity) {
    int n = weights.Length;
    int[, ] dp = new int[n + 1, capacity + 1];

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            dp[i, w] = dp[i - 1, w];           // exclude entirely
            if (weights[i - 1] <= w) {
                dp[i, w] = Math.Max(dp[i, w],
                    values[i - 1] + dp[i, w - weights[i - 1]]); // dp[i,...]
                // not dp[i-1,...] -- allows reusing item i again
            }
        }
    }
    return dp[n, capacity];
}

```

The only change from the 0/1 version is $dp[i, w - \text{weights}[i-1]]$ instead of $dp[i - 1, w - \text{weights}[i-1]]$ in the include branch — referencing the same item row means item i remains available for inclusion again at the reduced capacity, directly implementing “items can be reused” with this single, easy-to-miss index change.

ADVANCED — a disguised knapsack problem: partition a set into two subsets with minimum possible difference in sum:


```

int MinPartitionDifference(int[] nums) {
    int totalSum = nums.Sum();
    int n = nums.Length;
    bool[,] dp = new bool[n + 1, totalSum + 1]; // dp[i,s] = "achievable?"
    dp[0, 0] = true;

    for (int i = 1; i <= n; i++) {
        for (int s = 0; s <= totalSum; s++) {
            dp[i, s] = dp[i - 1, s]; // exclude
            if (s >= nums[i - 1]) {
                dp[i, s] |= dp[i - 1, s - nums[i - 1]]; // include
            }
        }
    }

    int best = int.MaxValue;
    for (int s = 0; s <= totalSum / 2; s++) {
        if (dp[n, s]) {
            best = Math.Min(best, totalSum - 2 * s); // one = s, other = rest
        }
    }
    return best;
}

```

This problem never mentions “weight,” “value,” or “capacity” explicitly, but it’s structurally identical to 0/1 knapsack: “capacity” becomes “target subset sum” (capped at half the total), each number is a single-use “item” with weight equal to its own value, and the include/exclude recurrence is unchanged — recognizing this disguised structure is exactly the pattern-recognition payoff this entire topic builds toward.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, 0/1 knapsack’s 2D table can often be space-optimized down to a single 1D array (a direct application of 11.7’s final conversion step), specifically by iterating the capacity dimension in decreasing order within each item’s pass — this ordering subtlety prevents a single array slot from being incorrectly updated using a value already modified earlier in the same item’s pass, which would effectively (and incorrectly) allow that item to be reused. There’s also a well-known further generalization called the multi-dimensional knapsack (multiple simultaneous capacity constraints) which extends the state to three or more dimensions, directly building on 11.5’s deeper-knowledge note about higher-dimensional DP generalizations.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is confusing the 0/1 and unbounded variants’ single critical index difference ($dp[i-1]$ vs $dp[i]$ in the include branch) — using the wrong one silently produces either an under-counted 0/1 answer or an over-counted, incorrect 0/1 answer. Another is not recognizing a disguised knapsack problem because it doesn’t use the literal weight/value/capacity vocabulary — missing that “partition into two subsets with minimum difference,” “can a subset sum to exactly target X,” and similar problems are all knapsack pattern instances wearing different surface clothing. A third: attempting the space-optimized 1D array version without correctly handling the capacity-iteration-direction subtlety — iterating capacity in increasing order for 0/1 knapsack accidentally permits reuse within the same item’s pass, a genuinely common and hard-to-spot bug.

10. SELF-CHECK

- Why does the 0/1 knapsack’s include branch reference $dp[i-1, w - \text{weight}]$ rather than $dp[i, w - \text{weight}]$ — what specifically would go wrong if you used the wrong one?

- In the minimum-partition-difference example, walk through, out loud, why “achievable subset sums” maps onto the knapsack pattern's “capacity” concept, even though the problem never mentions capacity explicitly.
- Why does space-optimizing 0/1 knapsack into a 1D array require iterating the capacity dimension in decreasing order, while the unbounded knapsack variant doesn't have this same requirement?

11.7 — Converting a brute-force recursive solution into a DP solution step by step

1. PLAIN-LANGUAGE GIST

This topic consolidates the entire section into one explicit, repeatable pipeline: starting from a brute-force recursive solution, add memoization (11.2) to get a working top-down DP solution, then convert that into tabulation (11.3) for an iterative bottom-up version, and finally, if the recurrence only ever depends on a small fixed window of prior states, compress the full table down into just a few variables for a space-optimized final version — four concrete stages, each one optional depending on what the specific problem and constraints actually demand.

2. REAL-WORLD ANALOGY

Picture refining a first rough draft of an essay through successive editing passes: the first draft (brute-force) gets the ideas down inefficiently and repetitively; a second pass (memoization) cuts out the literally repeated paragraphs by referencing earlier sections instead of rewriting them; a third pass (tabulation) reorganizes the whole piece into a cleaner, more linear structure; and a final pass (space optimization) trims away anything no longer needed once the structure is clear, keeping only what's essential. Each pass builds on the previous one's insight, and you can stop at whichever pass already satisfies your actual needs.

3. CONNECT TO PRIOR KNOWLEDGE

This topic is the explicit synthesis of everything from 11.1 through 11.6 — it doesn't introduce new mechanics, but rather names and orders the exact sequence of transformations those topics already demonstrated individually, giving you a deliberate, step-by-step procedure to follow under interview time pressure rather than needing to rediscover the right approach from scratch each time.

4. CORE MECHANISM (Simple → Intermediate)

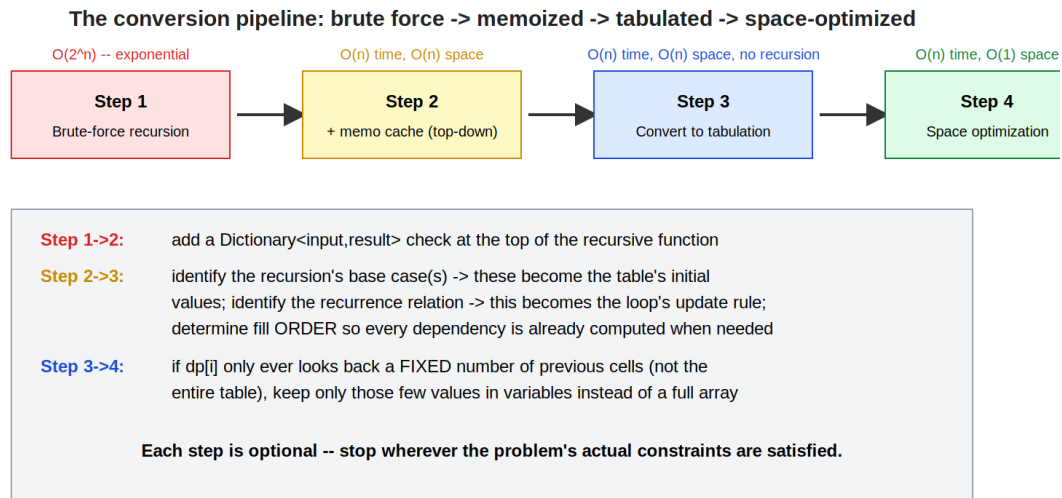
Simple model: Step 1: write the brute-force recursive solution directly from the problem's natural recursive structure (5.1's design process), without worrying about efficiency yet. Step 2: verify 11.1's two DP properties hold, then add a memo cache (11.2) — minimal code changes, immediate exponential-to-polynomial improvement if overlap existed. Step 3: convert the memoized recursive logic into an explicit table and loop (11.3), determining a valid fill order from the recurrence's dependencies. Step 4: if the recurrence only ever references a small, fixed number of previous table entries, replace the full array with just those few tracked variables, updated each iteration.

Next layer — the specific, mechanical sub-steps within each transition, and when to stop:

- **Step 1→2 specifics:** identify exactly what varies between recursive calls (the function's parameters) — this IS the cache key. Add a lookup at the top of the function and a store right before each return, with no other logic changes.
- **Step 2→3 specifics:** the recursive function's base case(s) become the table's initial seeded values. The recursive case's “combine smaller subproblems” logic becomes the loop body's update rule. The recursion's implicit dependency resolution must be replaced by an explicit, correctly-ordered loop that guarantees the same guarantee manually.
- **Step 3→4 specifics:** inspect the recurrence relation directly — if $dp[i]$ only ever reads from $dp[i-1]$ and $dp[i-2]$ (never anything further back), those are the only values that need to persist between iterations; everything else can be discarded immediately after use.
- **Knowing when to stop:** each step trades implementation effort for improved time/space characteristics — Step 2 is often “good enough” when recursion depth isn't a practical

concern and extra memory isn't tightly constrained; Step 4 is worth the added complexity specifically when memory is genuinely limited or when explicitly asked to optimize space. There's no obligation to always push all the way to Step 4.

5. VISUAL REPRESENTATION



The diagram shows the full four-stage pipeline as a left-to-right progression: brute-force recursion ($O(2^n)$ time) → add memoization ($O(n)$ time, $O(n)$ space) → convert to tabulation ($O(n)$ time, $O(n)$ space, no recursion) → space-optimize ($O(n)$ time, $O(1)$ space) — with the specific mechanical action required at each transition spelled out underneath.

6. WHEN TO USE / WHEN NOT TO USE

Apply this full pipeline when:

- You're solving a new DP problem under interview conditions and want a reliable, methodical process rather than trying to jump straight to an optimized solution and risking a subtle error in the rush.
- You're explicitly asked to optimize a working solution's time or space complexity as a follow-up — the pipeline's later steps are precisely the “next moves” to reach for.

Stop earlier in the pipeline when:

- A working, correct memoized (Step 2) solution is sufficient for the problem's actual constraints, and the added complexity of tabulation/space-optimization isn't justified by any real requirement.
- The recurrence doesn't have a clean “small fixed window” property needed for Step 4's space optimization — forcing that final compression onto a recurrence that genuinely needs the full table produces incorrect results, not just wasted effort.

7. C# EXAMPLES — Simple → Intermediate → Advanced

SIMPLE — the full four-step pipeline applied to Fibonacci, shown as four progressively refined versions:

```
// STEP 1: Brute force --  $O(2^n)$ 
long FibBrute(int n) => n <= 1 ? n : FibBrute(n - 1) + FibBrute(n - 2);

// STEP 2: Memoization --  $O(n)$  time,  $O(n)$  space
Dictionary<int, long> memo = new();
long FibMemo(int n) {
    if (n <= 1) return n;
    if (memo.ContainsKey(n)) return memo[n];
    return memo[n] = FibMemo(n - 1) + FibMemo(n - 2);
}

// STEP 3: Tabulation --  $O(n)$  time,  $O(n)$  space, no recursion
long FibTable(int n) {
    if (n <= 1) return n;
    long[] dp = new long[n + 1];
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
}

// STEP 4: Space-optimized --  $O(n)$  time,  $O(1)$  space
long FibOptimized(int n) {
    if (n <= 1) return n;
    long prev2 = 0, prev1 = 1;
    for (int i = 2; i <= n; i++) {
        long curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

INTERMEDIATE — applying Step 4's space optimization to House Robber (11.4), recognizing the same “only look back 2” property:

```
int HouseRobberOptimized(int[] houses) {
    int prev2 = 0, prev1 = 0;
    foreach (int house in houses) {
        int curr = Math.Max(prev1, prev2 + house);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

This collapses 11.4's full `dp[]` array down to just two tracked variables, since the recurrence only ever reads the two immediately preceding values — directly applying the same Step 4 pattern shown for Fibonacci to a different 1D DP problem, confirming the technique generalizes across the pattern category.

ADVANCED — applying Step 4's space optimization to a 2D problem (LCS, 11.5), compressing the full table down to just two rows:

```

int LCSSpaceOptimized(string s1, string s2) {
    int m = s1.Length, n = s2.Length;
    int[] prevRow = new int[n + 1];
    int[] currRow = new int[n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            currRow[j] = (s1[i - 1] == s2[j - 1])
                ? 1 + prevRow[j - 1]
                : Math.Max(prevRow[j], currRow[j - 1]);
        }
        (prevRow, currRow) = (currRow, prevRow); // swap for next row
    }
    return prevRow[n];
}

```

Since LCS's recurrence only ever reads from the immediately previous row and the current row's already-computed earlier column, the full $O(m \times n)$ table compresses down to just two $O(n)$ -sized rows, swapped after each outer iteration — a 2D instance of the exact same “only the last few states matter” insight that drove Fibonacci and House Robber's 1D space optimizations, scaled up to a row-at-a-time granularity instead of a single-value granularity.

8. DEEPER KNOWLEDGE (Gist Only)

At an expert level, this conversion pipeline is sometimes formalized as the general technique of “rolling array” optimization, a named pattern in competitive programming and algorithm engineering specifically for the Step 3→4 transition on 2D (or higher) DP tables — recognizing and applying it by name signals familiarity with the broader algorithmic toolkit beyond just solving the immediate problem at hand. There's also a worth-knowing caveat: not every problem's recovery requirements survive space optimization cleanly — if you need 11.3's path-reconstruction capability, aggressively compressing the table away (Step 4) can destroy the historical information needed for that reconstruction, meaning the choice of how far to push this pipeline sometimes depends on whether the problem needs just a value or also needs to explain how that value was achieved.

9. COMMON MISCONCEPTIONS AND MISTAKES

A common mistake is jumping straight to writing a space-optimized iterative solution from scratch, skipping the intermediate steps, and introducing subtle bugs in the process — methodically working through each pipeline stage catches errors that an attempted one-shot optimized solution often hides until they cause a wrong answer on a specific edge case. Another is over-optimizing a solution that didn't need it — converting a perfectly adequate memoized solution all the way to a space-optimized tabulated one when the problem's actual constraints never demanded that level of optimization, spending interview time on unnecessary work. A third: attempting Step 4's space optimization on a recurrence that doesn't actually have the required “small fixed window” property, forcing variables to track values that the recurrence genuinely needs from much further back, producing silently incorrect results once the compressed state no longer captures everything the calculation actually depends on.

10. SELF-CHECK

- Walk through, out loud, the specific mechanical change required at each of the three pipeline transitions — what exactly changes at each step?
- Why does House Robber's recurrence qualify for Step 4's space optimization, while a hypothetical recurrence that needed to reference $dp[i-5]$ would require keeping a larger, fixed-size window rather than just two variables?

- Why might you choose to stop at Step 2 (memoization) rather than pushing all the way to Step 4 (space optimization) for a given problem — what specific factors would make that the right call?